

普及组初赛模拟题（六）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. 八进制数 $(7042)_8$ 转化为十六进制数是()
 - A. $(3521)_{16}$
 - B. $(F22)_{16}$
 - C. $(E22)_{16}$
 - D. $(111000100010)_{16}$
2. 设栈 S 和队列 Q 初始状态为空，元素 $a_1, a_2, \dots, a_6$ 依次通过栈 S ，一个元素出栈后就进入队列 Q ，若出队的顺序分别是 $a_2, a_1, a_3, a_6, a_5, a_4$ ，则栈 S 的容量至少是()
 - A. 2
 - B. 5
 - C. 3
 - D. 4
3. 逻辑表达式（ ）的值与变量 A 的真假无关。
 - A. $(A \wedge B) \vee (\neg A \wedge B)$
 - B. $(A \vee B) \wedge \neg A$
 - C. $(A \vee B) \wedge \neg B$
 - D. $(A \vee B) \wedge \neg A \wedge B$
4. n 是一个三位数,那 n 的十位数为()。
 - A. $(n\%100)/10$
 - B. $(n/100)\%10$
 - C. $(n/100)\%100$
 - D. $(n\%10)/10$
5. 基于比较的排序时间复杂度的下限是(), 其中 n 表示待排序的元素个数。
 - A. $O(n^2)$
 - B. $O(n \log n)$
 - C. $O(n)$
 - D. $O(\log n)$
6. 完全二叉树共有 $2 \times N - 1$ 个结点，则它的叶节点数是（ ）。
 - A. $N - 1$
 - B. $2 \times N$
 - C. $2 \times N - 1$

◦ D. N

7. 45 和 30 的最小公倍数是()

◦ A. 45

◦ B. 30

◦ C. 90

◦ D. 180

8. 一棵 7 节点二叉树的中序遍历为 $ABDGE CF$, 先序遍历为 $DBACEGF$, 后序遍历为 ()

◦ A. $ABCDEF G$

◦ B. $DGBEFAC$

◦ C. $GBEACFD$

◦ D. $ABGEFCD$

9. 一个 n 个顶点的强连通图最少有几条边()

◦ A. n

◦ B. $n + 1$

◦ C. $n - 1$

◦ D. $n \times (n - 1)$

10. 若有如下程序段, 其中 s 、 a 、 b 、 c 均已定义为整型变量, 且 a 、 c 均已赋值 ($c > 0$)

```
1 | s = a;  
2 | for (b = 1; b <= c; b++) s = s - 1;
```

则与上述程序段功能等价的赋值语句是()。

◦ A. $s = a - b$;

◦ B. $s = s - c$;

◦ C. $s = a - c$;

◦ D. $s = b - c$;

11. 7 个人分乘两辆不同的汽车, 每辆车最多坐 4 人, 则不同的乘车方法数为()。

◦ A. 40

◦ B. 50

◦ C. 70

◦ D. 60

12. 以下关于图的不正确说法是()。

◦ A. 所有顶点的度数之和不一定等于边数的 2 倍

◦ B. 所有顶点的度数之和等于边数的 2 倍

◦ C. 在有向图中顶点的入度之和等于出度之和

◦ D. 任意一个图一定有偶数个度数为奇数的点

13. 定义一个数是“好的”: 当且仅当这个数是个六位数(允许有前导零), 并且里面含有数字 9, 那么符合“好的”条件的数的个数是 () 。

◦ A. 531441

- B. 1000000
- C. 99999
- D. 468559

14. 下列叙述中正确的是（ ）。

- A. 二叉树是线性结构
- B. 栈与队列是非线性结构
- C. 线性表是线性结构
- D. 线性链表是非线性结构

15. 现有八人排成一排照相，其中甲乙丙三人两两之间都不能相邻的排法有 [] 种

- A. $A(6, 3) \times A(5, 5)$
- B. $A(8, 8) - A(6, 6) \times A(3, 3)$
- C. $A(5, 3) \times A(3, 3)$
- D. $A(8, 8) - A(6, 4)$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外， 判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  const int mod = 2048;
4  long long c, n;
5  long long kasumi(long long x, long long mi) {
6      long long res = 1;
7      while (mi) {
8          if (mi & 1) {
9              res = (res * x) % mod;
10             }
11             x = (x * x) % mod;
12             mi >>= 1;
13         }
14         return res;
15     }
16     int main() {
17         cin >> n >> c;
18         if (n == 3) {
19             printf("%lld", c * (c - 1));
20             return 0;
21         }
22         long long ans = ((kasumi(c - 1, n) + (c - 1) *
23                             kasumi(-1, n)) % mod + mod) % mod;
24         cout << ans;
25         return 0;
26     }

```

16. 将第 9 行 `res = (res * x) % mod;` 和第 11 行 `x = (x * x) % mod;` 的括号去掉, 程序输出结果一定不变。[]
17. 将第 12 行的 `mi >>= 1` 改为 `mi /= 2`, 程序输出结果一定不变。[]
18. (2分) 若输入为 4 4, 则输出为 78。[]
19. 此程序的时间复杂度为 $O(\log n)$ 。[]
20. 若输入为 3 4, 则输出为 ()。
- A. 18
 - B. 19
 - C. 12
 - D. 8
21. 若输入为 2046 13 的返回值为 ()。
- A. 2024
 - B. 2
 - C. 12
 - D. 2022

阅读下面程序, 完成第 22 ~ 27 题。

```

1  #include<bits/stdc++.h>
2
3  using namespace std;
4  int main() {
5      int num = 0;
6      cin >> num; //保证num>= 100,且在int范围内
7      int max_primedivisor = 0;
8      int cnt = 1;
9      for (int i = 2; i * i <= num; i++) {
10         if (num % i == 0) {
11             int tmp = 1;
12             while (num % i == 0) num /= i, tmp++;
13             max_primedivisor = max(max_primedivisor, i);
14             cnt *= tmp;
15         }
16     }
17     max_primedivisor = max(max_primedivisor, num);
18     if (num > 1) cnt *= 2;
19     cout << max_primedivisor << " " << cnt << "\n";
20     return 0;
21 }

```

22. 代码中 `max_primedivisor = max(max_primedivisor ,num);` 这句话去掉对答案没有影响。[]
23. 当读入的 $num=p*q$ 其中 $p < q$, 且 p,q 为质数, 则 for 循环中 i 遍历到 q 时退出循环。[]
24. 该算法的最坏时间复杂度为()
- A. $O(\log num)$
 - B. $O(\sqrt{num})$
 - C. $O(num)$
 - D. $O(num\sqrt{num})$
25. 当读入 2021 时输出为()
- A. 43 2
 - B. 43 4
 - C. 47 2
 - D. 47 4
26. 当读入的数 $num = p * p * p * q * q * r * r * s * t$ 时,其中 $p < q < r < s < t$,且 p, r, q, r, s, t 均为质数, 则输出的第二个数()
- A. 144
 - B. 9
 - C. 12
 - D. 不确定
27. 在最好的情况下,时间复杂度为()
- A. $O(\sqrt{num})$
 - B. $O(num)$

- C. $O(num\sqrt{num})$
- D. $O(\log num)$

阅读下面程序，完成第 28 ~ 33 题。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3  #define maxn 110
4  struct node {
5      int r, c;
6      int A[maxn][maxn];
7  } G[maxn];
8  void Change(int id) {
9      int B[maxn][maxn];
10     for (int i = 1; i <= G[id].r; i++) {
11         for (int j = 1; j <= G[id].c; j++) {
12             B[j][i] = G[id].A[i][j];
13         }
14     }
15     swap(G[id].r, G[id].c);
16     for (int i = 1; i <= G[id].r; i++) {
17         for (int j = 1; j <= G[id].c; j++) {
18             G[id].A[i][j] = B[i][j];
19         }
20     }
21 }
22 void Add(int a, int b) {
23     for (int i = 1; i <= G[a].r; i++) {
24         for (int j = 1; j <= G[a].c; j++) {
25             G[a].A[i][j] += G[b].A[i][j];
26         }
27     }
28 }
29 void Subtract(int a, int b) {
30     for (int i = 1; i <= G[a].r; i++) {
31         for (int j = 1; j <= G[a].c; j++) {
32             G[a].A[i][j] -= G[b].A[i][j];
33         }
34     }
35 }
36 void Multiply(int a, int b) {
37     int Sum[maxn][maxn] = {0};
38     for (int i = 1; i <= G[a].r; i++) {
39         for (int j = 1; j <= G[b].c; j++) {
40             for (int k = 1; k <= G[a].c; k++) {
41                 Sum[i][j] = G[a].A[i][k] * G[b].A[k][j];
42             }
43         }
44     }
45     G[a].c = G[b].c;
46     for (int i = 1; i <= G[a].r; i++) {
47         for (int j = 1; j <= G[a].c; j++) {
48             G[a].A[i][j] = Sum[i][j];
49         }
50     }

```

```

51 }
52 void print1(int id) {
53     for (int i = 1; i <= G[id].r; i++) {
54         for (int j = 1; j <= G[id].c; j++) {
55             printf("%d%c", G[id].A[i][j], j < G[id].c ?
56                 ' ' : '\n');
57         }
58     }
59 }
60 void print2(char str[]) {
61     printf("%s\n", str);
62 }
63 int main() {
64     int n, m;
65     scanf("%d %d", &n, &m);
66     for (int k = 1; k <= n; k++) {
67         scanf("%d %d", &G[k].r, &G[k].c);
68         for (int i = 1; i <= G[k].r; i++) {
69             for (int j = 1; j <= G[k].c; j++) {
70                 scanf("%d", &G[k].A[i][j]);
71             }
72         }
73     }
74     for (int i = 1; i <= m; i++) {
75         int op, x, y;
76         scanf("%d%d%d", &op, &x, &y);
77         if (op == 0) {
78             Change(x);
79             Change(y);
80             if (i % 2) {
81                 print1(x);
82             } else {
83                 print1(y);
84             }
85         } else if (op == 1) {
86             if (G[x].r != G[y].r || G[x].c != G[y].c) {
87                 print2("error1");
88                 continue;
89             }
90             Add(x, y);
91             print1(x);
92         } else if (op == 2) {
93             if (G[x].r != G[y].r || G[x].c != G[y].c) {
94                 print2("error2");
95                 continue;
96             }
97             Subtract(y, x);
98             print1(y);
99         } else {
100             if (G[x].c != G[y].r) {
101                 print2("error3");

```

```

102         continue;
103     }
104     Multiply(x, y);
105     print1(x);
106 }
107 }
108 return 0;
109 }

```

提示：

矩阵转置：将第 i 行，变为第 i 列，如：

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

矩阵加减法：要保证两个矩阵的行数和列数相同，且相同位置相加减，如：

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix} + \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 2 & 6 & 10 \\ 6 & 10 & 12 \\ 10 & 14 & 18 \end{bmatrix}$$

矩阵乘法：矩阵 $A \times$ 矩阵 $B =$ 矩阵 C ，要保证 A 的列数和 B 的行数相同。则 $C_{i,j} = \sum_{k=1}^{\text{第 } i \text{ 行的元素个数}} A_{i,k} \times B_{k,j}$ ，如：

$$\begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 6 \\ 3 & 6 & 9 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} = \begin{bmatrix} 66 & 78 & 90 \\ 64 & 77 & 90 \\ 90 & 108 & 126 \end{bmatrix}$$

28. n, r, c 必须小于 110，否则程序可能会发生运行错误。[]

29. (2分) 第 85 ~ 88 行不能预防第 x 和第 y 个矩阵不能相加的情况。

30. 如果将第 41 行代码修正，则函数 `void Multiply(int a, int b)` 的功能是计算第 a 个矩阵乘以第 b 个矩阵的结果。[]

31. 第 10 行和第 11 行代码交换位置后，程序运行错误。[]

32. 第 41 行代码是否有误？如果有误，那么正确的代码应该为。[]

- A. 代码正确
- B. 代码有误；正确代码： `Sum[i][j] += G[a].A[i][j] * G[b].A[k][j];`
- C. 代码有误；正确代码： `Sum[i][j] += G[a].A[i][k] * G[b].A[k][j];`
- D. 代码有误；正确代码： `G[a].A[i][j] += G[a].A[i][k] * G[b].A[k][j];`

33. 若输入：

2 2
3 3
1 2 3
4 5 6
7 8 9
3 3
1 4 7
2 5 6
3 6 9
0 1 2
1 1 2

则正确输出为：

◦ A.

1 2 3
4 5 6
7 6 9
2 6 10
6 10 14
10 12 18

◦ B.

1 4 7
2 5 8
3 6 9
error1

◦ C.

1 4 7
2 5 8
3 6 9
66 64 90
78 77 108
90 90 126

◦ D.

1 4 7
2 5 8
3 6 9
2 6 10
6 10 14
10 12 18

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

输入月份 $m(1 \leq m \leq 12)$ ，按一定格式打印 2015 第 m 月的月历。

例如，2015 年一月的月历打印效果如下（第一列为周日）：

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

```

1  #include<iostream>
2
3  using namespace std;
4  const int dayNum[] = {-1, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
5  int m, offset, i;
6  int main() {
7      cin >> m;
8      cout << "S\tM\tT\tW\tT\tF\tS" << endl;
9      //'\t'为tab制表符
10     ①;
11     for (i = 1; i < m; i++)
12         offset = ②;
13     for (i = 0; i < offset; i++)
14         cout << '\t';
15     for (i = 1; i <= ③; i++) {
16         cout << ④;
17         if (i == dayNum[m] || ⑤ == 0)
18             cout << endl;
19         else
20             cout << '\t';
21     }
22     return 0;
23 }

```

1. ①处应填()

- A. offset = 0
- B. offset = 1
- C. offset = 3
- D. offset = 4

2. ②处应填()

- A. dayNum[i]
- B. offset + dayNum[i]
- C. (offset + dayNum[i]) % 7
- D. (offset + dayNum[i - 1]) % 7

3. ③处应填()

- A. m
- B. dayNum[m]
- C. offset
- D. offset + dayNum[i]

4. ④处应填()

- A. i
- B. i + 1
- C. i - 1

- D. `dayNum[i]`

5. ⑤处应填()

- A. `offset + i`
- B. `(offset + i) % 7`
- C. `offset + dayNum[i]`
- D. `(offset + dayNum[i]) % 7`

阅读下面题目，完成第 39 ~ 43 题。

给出一张 n 节点 m 条边的有向图，求出该图的一个拓扑排序，若无拓扑排序输出 -1

输入：

第一行两个正整数 n, m 表示点数和边数。

接下来 m 行，每行三个正整数 x, y 表示节点 $x \rightarrow y$ 之间有一条边。

输出：

一个拓扑序：按拓扑序输出点的编号。若拓扑序不唯一，输出任意一个均可。若无拓扑序，输出 -1 。

```

1  #include<algorithm>
2
3  #include<cstdio>
4
5  #include<vector>
6
7  #define N 200020
8  using namespace std;
9  int n, m;
10 vector < int > G[N];
11 int q[N], hd, tl;
12 int du[N];
13 int ans[N], tot;
14 void topo() {
15     hd = 1, tl = 0;
16     for (int i = 1; i <= n; i++)
17         if (__(1) __) q[++tl] = i;
18     while (hd <= tl) {
19         int u = q[hd++];
20         ans[++tot] = u;
21         for (int i = 0; __(2) __; i++) {
22             int v = G[u][i];
23             du[v]--;
24             if (!du[v]) __(3) __;
25         }
26     }
27     if (tot != n) puts("-1");
28     else {
29         for (int i = 1; i <= n; i++)
30             printf("%d ", ans[i]);
31     }
32 }
33 int main() {
34     scanf("%d%d", & n, & m);
35     for (int i = 1; i <= m; i++) {
36         int x, y;
37         scanf("%d%d", & x, & y);
38         __(4) __;
39         __(5) __;
40     }
41     topo();
42 }

```

39. 上述程序__(1)__中应该填写 ()

- A. du[i]
- B. q[i]
- C. hd <= tl
- D. !du[i]

40. 上述程序__ (2) __中应该填写 ()

- A. `i <= n`
- B. `i < n`
- C. `i < G[u].size()`
- D. `i <= G[u].size()`

41. 上述程序__ (3) __中应该填写 ()

- A. `q[++tl] = v`
- B. `q[tl++] = v`
- C. `q[++hd] = v`
- D. `q[hd++] = v`

42. 上述程序__ (4) __中应该填写 ()

- A. `G[y].push_back(x)`
- B. `G[x].push_back(y)`
- C. `G[x].push(y)`
- D. `G[y].push(x)`

43. 上述程序__ (5) __中应该填写 ()

- A. `G[y].push_back(x)`
- B. `G[y].push(x)`
- C. `du[y]++`
- D. `du[x]++`